CS683 PA1, Task 1: 2D Convolution, the Dhurandhar Way

Loop reordering, unrolling, cache tiling, SIMD, and their combination

Name	Roll number
Pranav	24B1074
Atharva	24B0996
Divyaansh	24B0981
Sufiyan	24B0935

Autumn 2026

1 Setup and methodology

Machine. All numbers in this report come from one Intel Core i7-13700HX (Raptor Lake, hybrid). We pinned every run to CPU 0, a P-core, with `taskset` and set the frequency governor to `performance`. `lscpu -C` reports L1-D 48 KB, 12-way, 64 B lines; L2 1.25 MB per P-core; L3 30 MB shared. The consumer Raptor Lake parts have AVX-512 fused off, so the SIMD experiments use 128-bit (xmm) and 256-bit (ymm) registers only.

Build. We used the pinned flags from the provided `Makefile` for every stage and did not change them: `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`. Auto-vectorisation is off, so any vector code in the binaries is ours. The provided harness (`main.cpp`) takes the median of 7 timed repetitions after 2 warm-ups, checks every stage against `conv_naive` (max abs error $< 10^{-3}$), and reports speedup as $t_{\text{naive}}/t_{\text{stage}}$ on the same input.

Profiling. Counters come from `perf stat` (`L1-dcache-load-misses`, `instructions`). The harness always runs the naive reference before the stage under test (once for the check, then 2 warm-ups and 7 reps), so we obtained per-call counts by subtraction: a naive-only run gives the cost of the reference calls, and we subtract that from the run that also contains the stage. MPKI is $10^3\times$ L1-D misses divided by the instructions of the stage being measured.

Correctness. All five stages pass the harness smoke tests. We also checked them on ragged shapes: 100×8 with $K = 7$, 33×16 with $K = 1$, 257×520 with $K = 9$, 1×8 with $K = 5$, and 2050×2056 with $K = 3$. No stage assumes power-of-two sizes. Tile loops clamp at the image edge and the SIMD stages keep a scalar tail.

How the stages relate to the naive code. The naive loop nest is `oy, ox, ky, kx`: each output pixel is finished before the next one starts, from a $K \times K$ window. That order already has good locality, since the window slides by one column, so tiling has nothing to fix in it. The reordered nest (`ky, kx, oy, ox`) is the form where the inner loop becomes a unit-stride sweep over a whole output row, and that sweep is what unrolling, tiling and SIMD each act on. `conv_unroll`, `conv_tile` and `conv_simd` therefore each start from the naive arithmetic with only that reorder applied and add

exactly one technique, so the speedup of each file measures that technique. `conv_optimized` is a different design that combines them (Section 5).

2 Task 1A: loop reordering and unrolling

2.1 What we changed and why

Reordering (`conv_reorder.cpp`). The kernel loops move outermost, the output is zeroed once, and for every tap (k_y, k_x) the whole image is swept with `out[oy][ox] += in[oy+ky][ox+kx] * w`. The motivation is that the naive inner loop over k_x runs only 3 or 5 times, so most of its cost is loop overhead, index arithmetic and reloading the weight. After the reorder the innermost loop is W iterations of one multiply-accumulate with unit stride on both `in` and `out`, which is what the hardware prefetchers and, later, the vector unit want. The weight w is loop-invariant and stays in a register.

Unrolling (`conv_unroll.cpp`). We unroll the column loop by 8. The harness guarantees W is a multiple of 8, and 8 floats is one AVX register, which keeps this stage in step with the SIMD stage later. The 8 partial sums live in a local array `a[8]` that the compiler keeps in registers across the whole k_x loop; the running value is read from `out` once per k_y and written back once. We had three things in mind. The 8 accumulators are independent, so a single accumulator's latency chain is broken and the core can overlap 8 multiply-adds. Loop control and indexing are amortised over 8 pixels. And `out` traffic drops from K^2 read-modify-writes per pixel to K, one per k_y, because the k_x loop accumulates in registers.

2.2 Effectiveness and the metric

We measure effectiveness as wall-clock speedup over `conv_naive` on the same input, because that is what a caller of the routine sees and what the harness grades. GFLOP/s appears alongside it; the FLOP count is identical for every stage, so it is the same information on an absolute scale. Instruction count and MPKI appear in later sections to explain results, not as the target.

Table 1: Reorder and unroll, speedup over naive (median of 7). Data: `task1d_sections234_fixed.txt`.

$K = 3$, $H = W$	256	512	1024	2048	4096
naive time (ms)	0.274	1.161	3.936	16.69	58.35
reorder speedup	1.19×	1.51×	1.57×	1.14×	0.78×
unroll speedup	3.06×	3.28×	3.42×	3.30×	2.44×
$K = 5$, $H = W$	256	512	1024	2048	
reorder speedup	1.84×	1.85×	1.37×	1.33×	
unroll speedup	4.13×	4.37×	3.31×	4.21×	

Reordering on its own is a small win that depends on size, and at 4096 it is a loss. While the image fits in L2/L3 (up to 2048×2048, a 16 MB output) the simpler inner loop gains 1.1 to 1.6×. At 4096^2 the output is 64 MB, more than L3, and the reordered code now makes $K^2 = 9$ full passes over `out` and `in` from DRAM where naive makes one. It drops to 0.78×. The README warns about exactly this, and it is the memory-bound behaviour that tiling (1B) exists to remove.

Unrolling is a steady $\approx 3\times$. Its three sources do not depend on cache behaviour: register accumulation across k_x (fewer `out` loads and stores), 8 independent dependency chains, and amortised loop overhead. It stays nearly flat across sizes and keeps most of its gain at 4096, where reorder collapses, because it makes only K passes over `out` instead of K^2. At $K = 5$ there is more arithmetic per pixel relative to memory traffic and the gain rises to as much as 4.4×.

3 Task 1B: cache tiling

3.1 Baseline profile

The L1 data cache is 48 KB, 12-way set associative, 64 B lines, 64 sets. For the naive convolution at $K = 3$ the L1-D MPKI is very low, between 0.13 and 0.31 across sizes 512 to 8192 (Table 2). Part of that is genuine reuse of the sliding window from L1. Most of it is the denominator: the naive kernel executes a very large number of instructions per pixel (K^2 multiply-adds, each with its own index arithmetic and weight reload). The reordered version does the same arithmetic in about 12× fewer instructions and shows what streaming the whole image K^2 times actually costs: 6 to 8 MPKI.

3.2 Tiled implementation and tile sweep

`conv_tile.cpp` blocks the output into TH × TW tiles. Inside each tile it runs the reordered nest (`ky, kx, oy, ox`), so all K^2 taps are applied to the tile while its $(\mathrm{TH} + K - 1) \times (\mathrm{TW} + K - 1)$ input patch and TH × TW output block are still in cache. Tile edges are clamped, so any H and W work. TH and TW are compile-time constants (`-DTH=...` `-DTW=...`), so the sweep recompiles instead of editing the file.

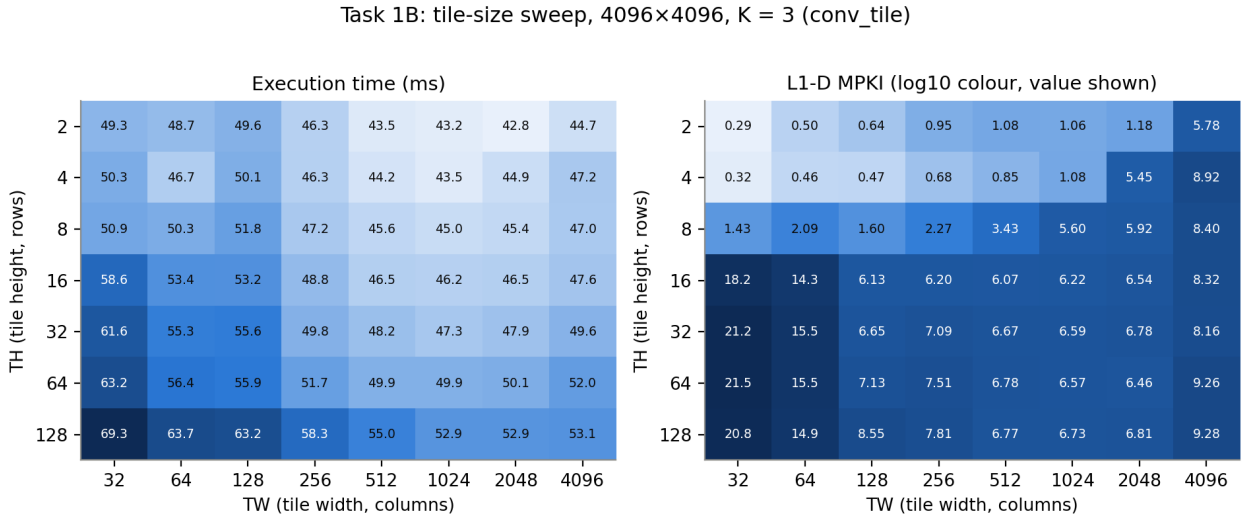

Figure 1: Execution time (left) and L1-D MPKI (right) of `conv_tile` over a 7 × 8 grid of tile shapes, 4096 × 4096, $K = 3$. Data: `task1b_full.txt` Section 2.

Figure 1 has three regions.

With TH ≤ 4 and TW ≤ 1024 the MPKI is low (0.3 to 1.1) and these are the fastest tiles (42.8 to 46.7 ms). The tile's working set is roughly (TH+2)·TW·4 B of input plus TH·TW·4 B of output, 24 KB at TH=4, TW=1024, which fits in L1 with room to spare.

At TW = 4096, the full width, MPKI jumps to 5.8 to 9.3. One row of input plus one of output at 4096 floats is already 32 KB, and the K^2 passes over TH such rows no longer fit.

At TH ≥ 16 the MPKI jumps to between 6 and 21 whatever TW is, even for tiny tiles: TH=16, TW=32 touches only 2.3 KB of data yet shows 18 MPKI. Capacity cannot explain that. What does is the row stride. At 4096 the stride is 16 KB, and the L1 set index comes from address bits 6 to 11, a 4 KB period, so every row of `out` lands in the same 12-way set, and so do the padded `in` rows (stride 16,392 B). Touching 16 output rows and 18 input rows of one tile column puts about 34 lines into a set with 12 ways, and the result is conflict misses. With TH ≤ 8 the per-set demand across the two arrays is at most about 18 lines and the aliasing is partly absorbed; at TH=4 it fits within

12 ways easily. This is the usual power-of-two-stride associativity problem, and it is why the fastest tiles here are short and wide rather than square.

Choice of tile. By execution time the optimum is the flat region TH=2 to 4, TW=1024 to 2048; five repeat runs at TH=2/TW=2048 and TH=4/TW=1024 land within 1 ms of each other. By MPKI the optimum is TH=2, TW=32 (0.30), but it is 15% slower, because very narrow tiles pay the K^2 loop-startup cost every 32 columns and hand the L2 streamer only 128 B runs. We ship TH=4, TW=1024. It sits in the fastest group at every image size we tested (Table 2), its 24 KB working set leaves half of L1 free for the halo and `out`, and its MPKI of about 1 is 6 to 7× below the untiled reorder.

3.3 MPKI and speedup versus matrix size

Table 2: L1-D MPKI and speedup over naive for the untiled and tiled kernels across image sizes, $K = 3$. Data: `task1b_full.txt` Section 4.

	L1-D MPKI					Speedup vs naive				
$H = W$	512	1024	2048	4096	8192	512	1024	2048	4096	8192
naive	0.196	0.139	0.131	0.197	0.311	1.00	1.00	1.00	1.00	1.00
reorder (untiled)	6.761	7.784	6.844	6.271	6.067	1.22	1.46	1.16	1.00	0.99
tile TH=4, TW=64	0.376	0.446	0.433	0.405	0.393	1.44	1.57	1.45	1.44	1.48
tile TH=4, TW=256	0.550	0.619	0.659	0.693	0.650	1.53	1.56	1.69	1.44	1.37
tile TH=4, TW=1024	0.594	0.743	1.026	1.063	1.014	1.60	1.78	1.77	1.54	1.37
tile TH=16, TW=512	5.536	5.466	5.856	6.150	6.638	1.91	1.45	1.58	1.42	1.49
tile TH=32, TW=128	6.407	6.620	6.784	6.647	7.858	1.48	1.33	1.39	1.24	1.19

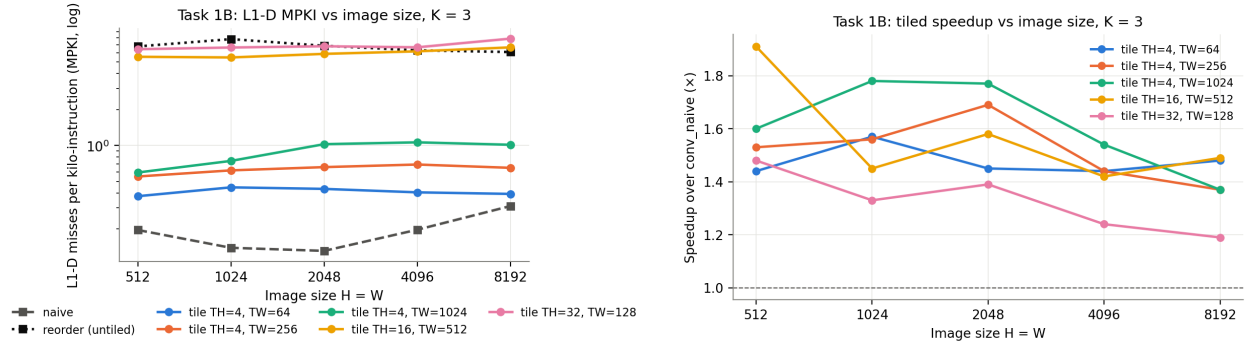

Figure 2: Left: L1-D MPKI versus image size for naive, untiled reorder and five tile shapes (log scale). Right: speedup of the tiled kernel over naive for the same tile shapes. $K = 3$.

3.4 Answers

Q1. Measured directly, MPKI goes up from about 0.2 (naive) to 0.4 to 1.0 (tiled, TH=4). That is the denominator again. MPKI is per thousand instructions, and the naive kernel spends about 12× more instructions per pixel on loop control, indexing and weight reloads, so its misses per instruction look small even though it has no cache-reuse advantage over the tiled code. The comparison that shows what tiling does to the cache is reorder versus tile, which have the same instruction count: MPKI falls from 6 to 8 down to 0.4 to 1.0, a 7 to 17× reduction. Each tile's input patch and output block are read K^2 times from L1 instead of the whole image being streamed K^2 times from L2, L3 or DRAM.

Q2. For a fixed good tile (TH=4) the MPKI barely moves with image size: 0.38 to 0.45 at TW=64, 0.59 to 1.06 at TW=1024. That is what a working set bounded by the tile rather than by the image looks like; the cache sees the same 24 KB problem whatever H and W are. MPKI grows with TW because longer rows mean fewer rows resident in L1, so the halo rows get fetched again

more often. Across tile shapes the TH threshold from the sweep dominates: TH $\geq$ 16 gives 5.5 to 7.9 MPKI at every size, no better than not tiling, because set conflicts rather than capacity decide the outcome. The naive kernel's own MPKI rises from 0.13 to 0.31 with size because at 8192 the K input rows and the output row, 32 KB each, no longer all fit in L1 together.

Q3. Yes. With TH=4, TW=1024 the tiled kernel is 1.37 to 1.78× faster than naive and 1.1 to 1.7× faster than the untiled reorder at every size. Tiling turns the K^2 image-wide streams into passes over L1-resident data, so the reorder's collapse at 4096 (0.78 to 1.00×) disappears and the tiled speedup holds at 1.4 to 1.5× even with a 256 MB output. The gain stops at about 1.7× because once the data comes from L1 the kernel is limited by scalar issue: every pixel-tap still costs a load, a multiply-add and a store of `out`. That is a compute-side limit, which tiling cannot touch, and it is the reason for 1C and 1D. One more observation: the speedup ranking and the MPKI ranking of tile shapes disagree. TW=64 has the fewest misses, TW=1024 the best time. An L1 miss that hits L2 costs about 14 cycles and is hidden by out-of-order execution and the L2 streamer when the run is long, so a lower MPKI is not worth K^2 extra loop restarts every 64 columns.

4 Task 1C: SIMD

4.1 Implementation

`conv_simd.cpp` vectorises the reordered nest. For each tap the weight is broadcast once with `_mm256_set1_ps`, and the row loop handles 8 output columns per iteration: an unaligned load of the input row (`_mm256_loadu_ps` at `in+ox+kx`), an unaligned load of the current output, one `_mm256_fmadd_ps`, and an unaligned store. A scalar tail covers W mod 8; the harness never takes it, but we kept it. The 128-bit variant is the same code with the `__m128` type and the `_mm_*` intrinsics, selected at compile time with `-DSIMD_W=128`. We checked both builds by disassembly: the 256-bit binary contains `ymm` FMAs, the 128-bit one contains none. AVX-512 is not available on this processor.

4.2 Instruction count and speedup

Table 3: Instructions retired per call (perf, subtracted) and speedup over naive for the SIMD kernel by width. Data: `task1c_full.txt` Section 4.

K	$H = W$	naive instr.	SIMD-128 instr.	SIMD-256 instr.	red. 128 / 256	speedup 128 / 256
3	1024	9.96×10^7	1.51×10^7	8.01×10^6	6.61 / 12.44	2.50 / 2.49
3	2048	3.97×10^8	6.03×10^7	3.19×10^7	6.59 / 12.46	1.92 / 2.18
3	4096	1.59×10^9	2.41×10^8	1.28×10^8	6.60 / 12.46	1.21 / 1.36
3	8192	6.36×10^9	9.62×10^8	5.10×10^8	6.61 / 12.48	1.04 / 1.13
5	1024	2.15×10^8	4.05×10^7	2.09×10^7	5.31 / 10.29	2.21 / 2.72
5	2048	8.59×10^8	1.62×10^8	8.29×10^7	5.32 / 10.37	2.03 / 2.19
5	4096	3.44×10^9	6.45×10^8	3.31×10^8	5.33 / 10.39	1.31 / 1.38
5	8192	1.38×10^{10}	2.58×10^9	1.32×10^9	5.33 / 10.41	1.29 / 1.32

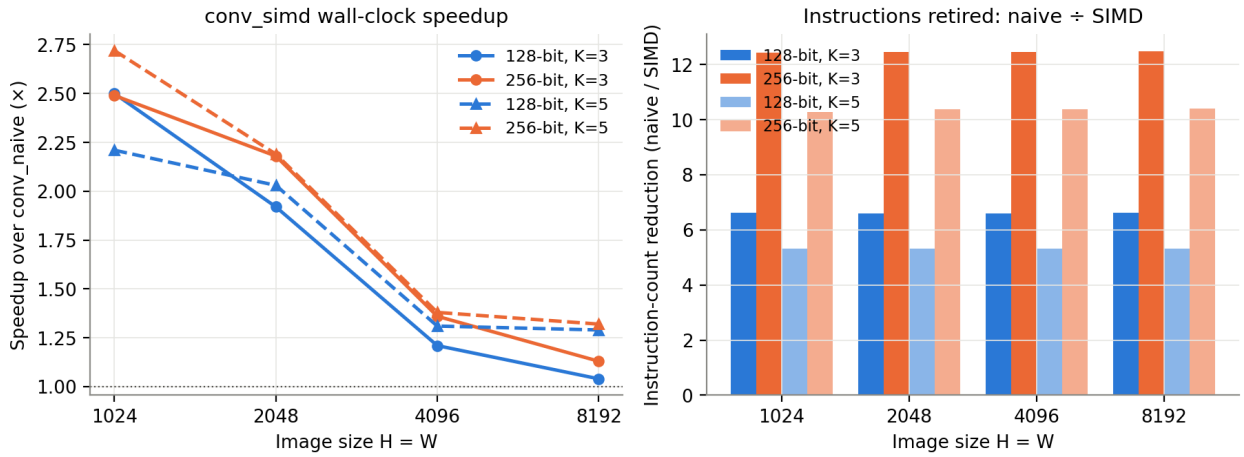

Figure 3: Left: speedup of `conv_simd` over naive versus image size for 128-bit and 256-bit builds. Right: instruction-count reduction factor. The 256 and 512 sizes run in well under a millisecond and were left out as too noisy.

4.3 Answers

Q1. Going from naive to 256-bit SIMD cuts retired instructions by 12.4 to 12.5× at $K = 3$ and 10.3 to 10.4× at $K = 5$; the 128-bit build gets 6.6× and 5.3×. The factor is constant across sizes to three digits, as it should be, since instruction count depends on the work and not on the cache. It is larger than the lane count (8 or 4) for two reasons. The SIMD kernel is built on the reordered loop, which already removes the naive code's per-pixel weight reload and 2-D index arithmetic, worth about 1.5× by itself. And the FMA fuses the multiply and the add that naive issues separately. The 256-bit build executes exactly half the instructions of the 128-bit build (12.44 / 6.61 = 1.88), which confirms the width switch took effect. At $K = 5$ the reduction is a little smaller because the per-row overhead (loop control, tail check) is spread over the same number of taps per row while the naive code amortises its own overhead better.

Q2. Yes, but much less than the instruction count suggests: 2.5× at 1024 falling to 1.1× at 8192 for 256-bit, and the 128-bit build is only 0 to 15% behind despite executing twice as many instructions. So the kernel is not instruction-bound. It is bound by memory bandwidth. Every 8-wide FMA needs two 32-byte loads and one 32-byte store, 12 B of traffic for every 2 FLOPs, and once the arithmetic is vectorised the load/store units and, for large images, DRAM are the wall. The decay with size is the reorder's K^2 image-wide passes again: the SIMD kernel is not tiled, so at 4096 and 8192 those passes come from DRAM and the vector unit waits. Where the data stays in L2 (1024) the vector unit's benefit is visible. Comparing with Table 2, un-tiled SIMD is slower than scalar tiling at 4096 and 8192, which is a good reason to combine the two (Task 1D).

Q3. The intrinsics we used, and why:

- `_mm256_loadu_ps` / `_mm_loadu_ps`. The input pointer is offset by $k_x = 0, 1, 2, \ldots$ floats, so it is unaligned for all but one tap and an aligned load would fault. On this core an unaligned load that stays inside a cache line costs the same as an aligned one.
- `_mm256_set1_ps`. Broadcasts one weight to all lanes. It is loop-invariant, hoisted out of the row loops, and executes only K^2 times per call.
- `_mm256_fmadd_ps`. One instruction and one rounding for `in` $\cdot\, w +$ `out`. It halves the arithmetic instruction count compared with separate `mul_ps` and `add_ps`, and it is why the pinned flags include `-mfma`. It also agrees slightly better with the naive reference, which `-O2` with FMA available contracts to the same fused operation.
- `_mm256_storeu_ps`. Unaligned store of the updated output vector, for the same reason as the load.

We did not use the double-precision `_pd` variants shown in the assignment appendix because the

data is `float32`; the `_ps` family gives twice the lanes per register.

5 Task 1D: combining the techniques

5.1 Design of `conv_optimized`

The combined kernel is not the reordered nest with SIMD added. It is a register-blocked design that takes one thing from each earlier stage.

From 1A (unroll), register accumulation. The loop order is `ty, tx, oy, ox, ky, kx`: for a strip of $8 \cdot \text{NACC}$ output columns all K^2 taps accumulate in `NACC __m256` registers, and the output is stored once. The output array is written a single time per call rather than K^2 times (reorder, tile, simd) or K times (unroll). This removes the load/store traffic that made 1C memory-bound.

From 1C, SIMD. Every tap is one `_mm256_fmadd_ps` with a broadcast weight and an unaligned input load, 8 outputs per instruction.

From unrolling, ILP. `NACC`=2 independent accumulators per row, 16 columns per iteration, so two FMA dependency chains are in flight. The FMA unit has a 4-cycle latency and can start two per cycle.

From 1B, tiling. The output is blocked into TH×TW tiles (16×512) so that the $\text{TH} + K - 1$ input rows a tile touches stay in L1/L2 while the row loop sweeps them K times.

All three parameters are compile-time macros and come from a 36-point sweep (Table 4). The kernel handles any H, W, K with an 8-wide tail and then a scalar tail.

Table 4: Best time over the TH×TW grid for each accumulator count, 4096×4096, $K = 3$ (`task1d_sweep.txt`).

NACC	1	2	4	8
best time (ms)	9.69	**7.23**	12.93	10.87
speedup vs naive	6.6×	**9.5**×	5.3×	6.4×

NACC=1 leaves the FMA latency exposed, with a single chain. NACC=2 hides it. NACC=4 and 8 are slower, not faster: each tap now needs 4 or 8 unaligned 32-byte loads, and with 2 load ports (3 on Raptor Cove, but with penalties for loads that cross a line) the loop becomes load-port-bound, while wider strips also push the halo out of L1. This is a measured sign that the combined kernel is load-bound rather than FMA-bound. TH=16 was best across 2048, 4096 and 8192. TW between 256 and 1024 is within noise, because the tile only has to keep input rows hot; with register accumulation `out` is no longer re-read.

5.2 Results

Table 5: Graded run on the Intel machine (`task1d_full_postfix.txt`): 2048×2048, $K = 3$, and the ungraded $K = 5$ table.

	$K = 3$ (graded)			$K = 5$		
stage	time (ms)	GFLOP/s	speedup	time (ms)	GFLOP/s	speedup
naive	19.25	3.92	1.00×	57.62	3.64	1.00×
reorder	19.63	3.85	0.98×	44.78	4.68	1.29×
unroll	7.40	10.20	2.60×	13.22	15.87	4.36×
tile	12.68	5.96	1.52×	35.62	5.89	1.62×
simd	11.18	6.75	1.72×	38.29	5.48	1.50×
optimized	**2.08**	**36.25**	**9.25**×	**3.49**	**60.05**	**16.50**×

The harness gives this run 40/40 for correctness and 60/60 for speed (9.25× clears the 8× top tier).

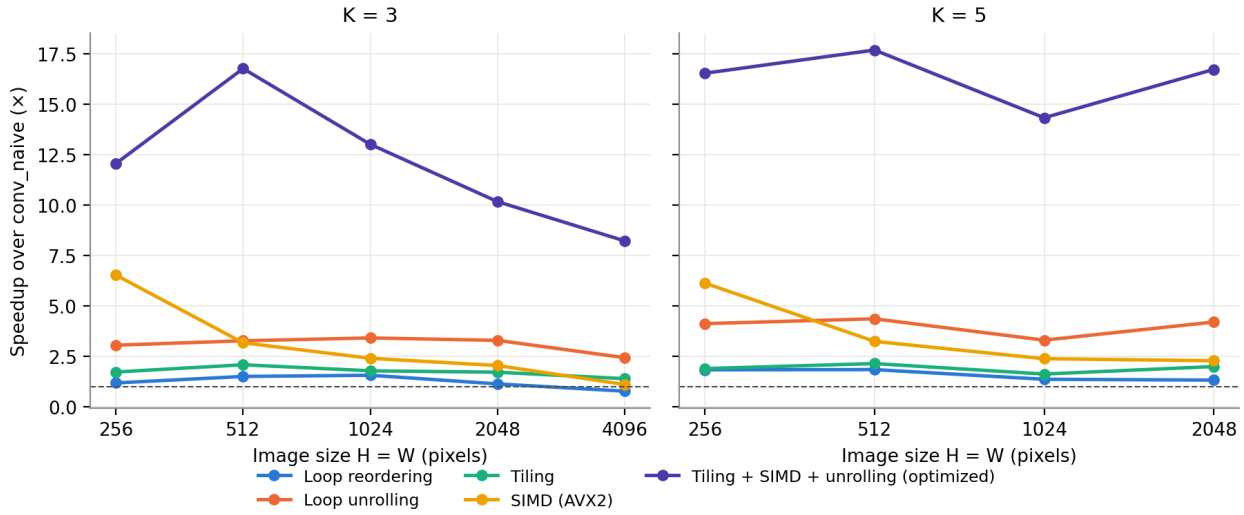

Figure 4: Figure 1.1 equivalent: speedup over naive of each technique versus image size, $K = 3$ (left) and $K = 5$ (right). Data: `task1d_sections234_fixed.txt`.

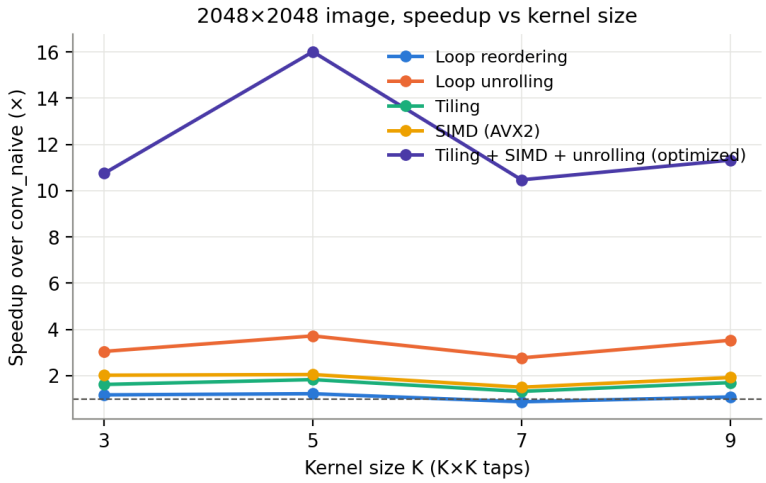

Figure 5: Speedup over naive versus kernel size at 2048×2048.

5.3 Why the combination is more than the sum

At 2048^2, $K = 3$ the best single techniques give 3.3× (unroll), 2.1× (SIMD) and 1.7× (tile). The combined kernel gives 10.2× in the same run and 9.25× in the graded run. That is more than the sum of the individual gains ($2.3 + 1.1 + 0.7 = 4.1$ on top of 1) and close to their product. Each technique removes a different limit, and each limit hides the next one.

SIMD on its own (1C) stops at about 2× because the reordered nest is memory-bound; the vector unit waits on the K^2 output read-modify-writes. Register accumulation removes those passes, but a scalar version of it (unroll, 1A) then runs into scalar FMA throughput at about 3×. Putting SIMD on the register-accumulated form removes that compute limit, and the next one to appear is the re-reading of input rows; tiling keeps those rows in L1, and NACC=2 hides the FMA latency.

With all four in place the kernel runs at 36 to 73 GFLOP/s and is limited by the load ports, which is what the NACC sweep showed. Figures 4 and 5 show the combined kernel keeping an 8 to 17× margin at every size and kernel width, while every single technique slides towards 1× at

4096 as the image leaves L3. The fall from 16.8× at 512 to 8.2× at 4096 is DRAM bandwidth on the one unavoidable pass over input and output. At $K = 5$ and $K = 7$ there is more arithmetic per byte and the speedup rises to 16 to 17×. The $K = 7$ dip in Figure 5 comes from the naive baseline running unusually fast in that run (6.4 GFLOP/s against 4.5 to 5.1 elsewhere), not from the optimized kernel slowing down; it stays between 51 and 73 GFLOP/s.

6 Summary

Reordering gives 1 to 1.5× and unrolling about 3× (1A). Tiling (1B) removes the reorder's image-wide passes, holds 1.4 to 1.8×, and cuts L1-D MPKI 7 to 17× against the untiled loop. We chose TH=4, TW=1024 because short, wide tiles avoid the power-of-two-stride set conflicts that make TH $\geq$ 16 behave as if there were no tiling at all. SIMD (1C) cuts instructions 12.5× but buys only 1.1 to 2.5× because the un-tiled loop is bandwidth-bound. Combining register accumulation, AVX2 FMA, two independent accumulators and output tiling (1D) reaches 9.25× on the graded workload and 8 to 17× across sizes and kernel widths, more than the sum of the individual gains, because each technique removes the limit that stopped the previous one. We did not use software prefetching anywhere in Task 1, as required.